

Sistem de Procesare Stereo și Generare Video Anaglif 3D

Student A

Student B

Cuprins

1	Introducere	2
2	Fundamentare teoretică	2
2.1	Principiul disparității și adâncimii	2
2.2	Funcții de cost pentru corespondența pixelilor	2
2.3	Filtre de netezire	3
3	Proiectare și implementare	3
3.1	Arhitectura generală	3
3.2	Pipeline de procesare	4
3.3	Algoritmul de calcul al adâncimii (SAD)	4
3.4	Sinteza anaglifei	5
3.5	Optimizări de performanță	5
3.6	Modul de utilizare	5
4	Contribuții individuale	5
4.1	Moldovan Flavius	5
4.2	Sur Patrick	5
5	Rezultate experimentale	6
5.1	Secvențe de test	6
5.2	Capturi de ecran ilustrative	7
6	Limitări	7
7	Concluzii	8
A	Codul sursă complet	9
A.1	Filtrul Median 5×5	9
A.2	Filtrul Gaussian 5×5	9
A.3	Calculul Hărții de Adâncime (SAD)	10
A.4	Generarea Anaglifei 3D	10
A.5	Funcția <code>main()</code> – meniu și pipeline principal	11

1 Introducere

Contextul temei:

Viziunea stereoscopică reprezintă o ramură fundamentală a viziunii computerizate, având ca scop extragerea informațiilor de adâncime (3D) din imagini bidimensionale capturate din unghiuri ușor diferite [2]. Imaginile anaglife reprezintă una dintre cele mai vechi și accesibile metode de vizualizare tridimensională, folosind ochelari cu filtre de culori complementare (Roșu–Cyan).

Problemele de rezolvat:

Generarea unei imagini anaglife prin simpla suprapunere a canalelor de culoare din două imagini brute produce artefacte vizuale și oboseală oculară (*eye strain*), în special atunci când distanța dintre camere (baseline) este mare. Soluția constă în calcularea unei hărți de adâncime (*Depth Map*) care controlează deplasarea sintetică a pixelilor, generând un efect 3D confortabil. Calculul disparității pixel-cu-pixel este, însă, costisitor din punct de vedere computațional, impunând tehnici de optimizare.

Obiectivele propuse:

- Implementarea unui algoritm eficient de tip Block Matching (SAD) pentru calcularea disparității [2].
- Optimizarea filtrelor de netezire (Median și Gaussian) pentru execuție accelerată [1].
- Generarea unei anaglife sintetice modulate de harta de adâncime, cu factor de confort controlabil.
- Procesarea secvențelor video cadru cu cadru și salvarea rezultatului în format AVI [3].

2 Fundamentare teoretică

2.1 Principiul disparității și adâncimii

Extragerea adâncimii se bazează pe principiul triangulației geometrice [2]. Disparitatea reprezintă diferența de coordonate orizontale ale aceluiași punct 3D proiectat pe două plane de imagine stânga–dreapta. Adâncimea Z este invers proporțională cu disparitatea d :

$$Z = \frac{f \cdot B}{d} \quad (1)$$

unde f este distanța focală a camerei, iar B este distanța fizică (baseline) dintre camere.

2.2 Funcții de cost pentru corespondența pixelilor

Pentru găsirea corespondenței se pot folosi mai multe funcții de cost [2]:

- **SSD** (Sum of Squared Differences) – sensibilă la zgomot, complexitate $O(n^2)$.
- **NCC** (Normalized Cross-Correlation) – robustă la variații de iluminare, dar lentă.

- **SAD** (Sum of Absolute Differences) – eficientă computațional, folosind doar operații aritmetice de bază.

S-a optat pentru SAD datorită eficienței sale pe CPU [1]. Costul SAD pentru o fereastră $(2w+1) \times (2w+1)$, disparitate d și coordonate (x, y) este:

$$SAD(x, y, d) = \sum_{u=-w}^w \sum_{v=-w}^w |I_L(x+u, y+v) - I_R(x+u-d, y+v)| \quad (2)$$

unde I_L și I_R sunt intensitățile din imaginile stânga, respectiv dreapta. Disparitatea optimă minimizează această funcție de cost.

2.3 Filtre de netezire

Harta de adâncime brută conține zgomot semnificativ și necesită post-procesare [1].

- **Filtrul Median 5×5** – înlocuiește fiecare pixel cu mediana vecinătății sale; eficient în eliminarea zgomotului impulsiv (*salt-and-pepper*).
- **Filtrul Gaussian 5×5** – aplică o medie ponderată cu kernel-ul Gaussian discret, producând o netezire uniformă a suprafețelor de adâncime.

3 Proiectare și implementare

3.1 Arhitectura generală

Aplicația a fost dezvoltată în C++ cu biblioteca OpenCV [3] pentru manipularea imaginilor și salvarea video. Paralelizarea la nivel de rând a fost realizată cu directive OpenMP [4], maximizând utilizarea procesorului multi-core.

3.2 Pipeline de procesare

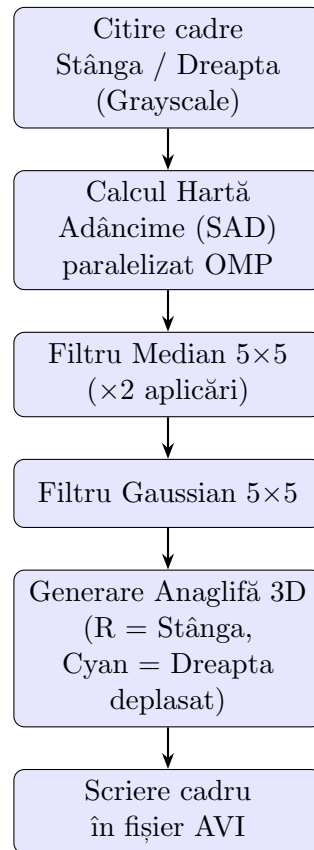


Figura 1: Pipeline-ul de procesare pentru un cadru stereo.

Pașii pipeline-ului (Figura 1) sunt:

1. **Citire cadre** – imaginile Stânga și Dreapta sunt încărcate în format grayscale.
2. **Calcul Hartă de Adâncime (SAD)** – se aplică Block Matching cu fereastră 11×11 și disparitate maximă 32 px (vezi Anexa A.3).
3. **Filtru Median 5×5 ($\times 2$)** – se aplică de două ori consecutiv pentru eliminarea zgomotului impulsiv (Anexa A.1).
4. **Filtru Gaussian 5×5** – netezire suplimentară a suprafețelor de adâncime (Anexa A.2).
5. **Generare Anaglifă 3D** – pixel-ii sunt mapați cu deplasare sintetică bazată pe harta de adâncime (Anexa A.4).
6. **Scriere în AVI** – cadrul color este adăugat în fișierul video final.

3.3 Algoritmul de calcul al adâncimii (SAD)

Implementarea SAD (Anexa A.3) iterează rândul-coloana cu paralelizare OMP pe rânduri. Pentru fiecare pixel, se caută fereastra de 11×11 cu costul minim pe intervalul de disparitate $[0, 32]$ px. Indexul ferestrei optime este normalizat la $[0, 255]$ și stocat în harta de adâncime.

3.4 Sinteza anaglifiei

Anaglifa nu realizează o simplă suprapunere, ci calculează o nouă coordonată orizontală pentru ochiul drept bazată pe harta de adâncime normalizată (Anexa A.4):

$$x_{\text{drept}} = x - \left\lfloor \frac{\text{Depth}(y, x) \cdot D_{\text{max}} \cdot C_{\text{factor}}}{255} \right\rfloor \quad (3)$$

unde $D_{\text{max}} = 32$ este disparitatea maximă (pixeli), iar $C_{\text{factor}} = 0.2$ este coeficientul de scalare a efectului 3D pentru prevenirea oboselii oculare.

3.5 Optimizări de performanță

- Alocările dinamice din buclele interioare au fost eliminate; filtrul median folosește un vector static `uchar valori[25]`.
- Toate buclele de rânduri sunt prefixate cu `#pragma omp parallel for [4]`.
- Scriere directă cu `.at<uchar>` evită overhead-ul iterator.

3.6 Modul de utilizare

La rulare, aplicația afișează un meniu interactiv cu 7 secvențe de test. Utilizatorul introduce o cifră (1–7); aplicația procesează folderele corespunzătoare, afișează în timp real harta de adâncime și anaglifa, și salvează rezultatul `.avi` în directorul curent.

4 Contribuții individuale

Proiectul a fost realizat în echipă de două persoane, cu diviziunea responsabilităților descrisă mai jos.

4.1 Moldovan Flavius

- **Filtrul Median 5×5** – implementare cu vector static `uchar[25]`, sortare in-place și paralelizare OMP (Anexa A.1).
- **Calculul Hărții de Adâncime (SAD)** – proiectarea și implementarea algoritmului Block Matching, alegerea parametrilor ($w = 11$, $D_{\text{max}} = 32$) și normalizarea rezultatelor (Anexa A.3).
- Integrarea modulelor de citire a fișierelor și gestionarea erorilor de I/O din `main()` (Anexa A.5).

4.2 Sur Patrick

- **Filtrul Gaussian 5×5** – implementare cu kernel discret normalizat ($c = 273$) și paralelizare OMP (Anexa A.2).
- **Generarea Anaglifiei 3D Sintetice** – proiectarea formulei de deplasare sintetică modulată de harta de adâncime, alegerea factorului de confort $C_{\text{factor}} = 0.2$ (Anexa A.4).

- Configurarea pipeline-ului video: inițializare **VideoWriter**, bucla principală de procesare cadru-cu-cadru și meniul interactiv din `main()` (Anexa A.5).

5 Rezultate experimentale

5.1 Secvențe de test

Sistemul a fost testat pe 7 secvențe video stereoscopice distincte, procesate cadru cu cadru. Tabelul 1 sintetizează comportamentul hărții de adâncime pentru fiecare test.

Tabela 1: Rezultate pe secvențele de test.

Secvență	Caracteristici scenă	Comportament hartă adâncime
Test 1 – Spiderman	Contrast ridicat, mișcare rapidă, explozii.	Obiectul central detectat corect; apare zgomot în zonele cu explozii și mișcare foarte rapidă.
Test 2 – Rollercoaster	Detalii geometrice fine, iluminare scăzută.	Șinele și elementele din prim-plan bine segmentate; fundalul întunecat induce pete inconsistente alb-negru.
Test 3 – Ball	Element „pop-out” (minge în mișcare directă spre cameră).	Gradient de adâncime clar vizibil: pe măsură ce mingea se apropie, nuanțele de gri devin tot mai deschise.
Test 4 – Helicopter	Obiecte zburătoare suprapuse pe fundal arhitectural static.	Segmentare corectă a planurilor: elicopterul apare în tonuri deschise, fundalul static mapat negru (îndepărtat).
Test 5 – Bunny	Animatie 3D, personaje clare, fundal vegetal complex.	Iepurele definit bine; complexitatea frunzișului generează pete alb-negru în fundal.
Test 6 – Butterfly	Fluturi cu mișcare fină, fundal uniform difuz.	Contururile aripilor sunt evidențiate.
Test 7 – Vettel (Racing)	Mașină de Formula 1, mișcare laterală rapidă, pistă cu marcaje.	Mașina și marcajele pistei sunt detectate corespunzător; viteza mare introduce artefacte de tip „ghosting” pe margini.

5.2 Capturi de ecran ilustrative

Figurile 2–4 prezintă un cadru reprezentativ din secvența Test 3 (Ball), ilustrând cele trei etape vizibile ale pipeline-ului.



Figura 2: Imagine de intrare – cadru stânga (grayscale), Test 3 – Ball.

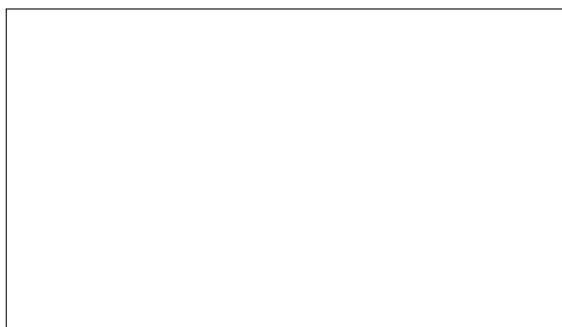


Figura 3: Hartă de adâncime generată (SAD + filtrare), Test 3 – Ball. Zonele deschise (alb) indică obiecte apropiate; zonele închise (negru) indică fundal îndepărtat.

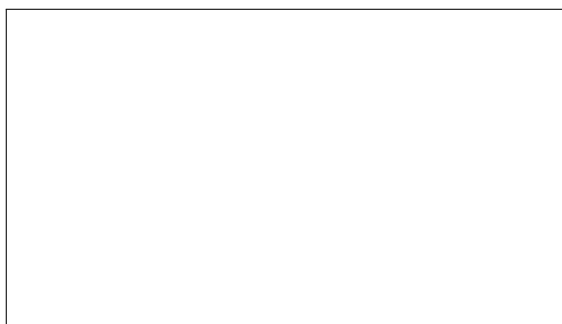


Figura 4: Anaglifă 3D sintetică generată (canal Roșu = Stânga, canal Cyan = Dreapta deplasat), Test 3 – Ball. Vizualizabilă cu ochelari Roșu-Cyan.

6 Limitări

- **Fereastră de căutare fixă** – dimensiunea 11×11 este un compromis: o fereastră mai mare produce efectul de dilatare a obiectelor (*fattening effect*), iar una mai mică introduce zgomot excesiv.

- **Sensibilitate la mișcare rapidă** – SAD este un algoritm static; cadrele cu mișcare foarte rapidă (Test 1 – Spiderman, Test 7 – Vettel) introduc artefacte de tip *ghosting* pe marginile obiectelor.
- **Dependența de textură** – algoritmul eșuează pe suprafețele uniforme (cer, fundal monocrom), generând disparitate incertă sau zgomot aleatoriu.
- **Procesare exclusiv pe CPU** – paralelizarea cu OpenMP este limitată la resursele CPU; rezoluțiile HD/4K nu pot fi procesate în timp real pe hardware-ul curent.
- **Borduri negre (border effect)** – primele și ultimele **offset** linii/coloane rămân negre deoarece fereastra 5×5 (sau 11×11) nu poate fi aplicată la marginea imaginii.
- **Format de intrare restrictiv** – aplicația acceptă doar imagini **.bmp** cu denumire sortabilă lexicografic; alte formate (JPEG, PNG) nu sunt gestionate implicit.

7 Concluzii

Aplicația dezvoltată este pe deplin funcțională, atingând toate obiectivele propuse. Pipeline-ul optimizat cu OpenMP permite redarea și salvarea secvențelor video fluent pe CPU. Harta de adâncime este calculată corect și dictează direct efectul stereoscopic al anaglifei sintetice.

Contribuții rezumative:

- Redesign-ul filtrelor spațiale pe memorie statică (eliminarea alocărilor dinamice din buclele interioare).
- Implementarea SAD parametrizată, cu normalizare pe intervalul $[0, 255]$.
- Conceptul de „Anaglifă Sintetică” – modularea pixelilor prin harta de adâncime locală în locul citirii directe a imaginii drepte, cu factor de corecție C_{factor} .

Direcții de dezvoltare:

Pentru versiunile viitoare se propun: (1) implementarea Semi-Global Block Matching (SGBM) [2] pentru suprafețe lipsite de textură; (2) portarea SAD pe GPU prin CUDA pentru procesare HD/4K în timp real (>60 FPS); (3) suport pentru formatul PNG/JPEG și detectarea automată a fps-ului din metadatele secvenței.

Bibliografie

- [1] Bradski, G., & Kaehler, A. (2008). *Learning OpenCV: Computer vision with the OpenCV library*. O'Reilly Media, Inc.
- [2] Scharstein, D., & Szeliski, R. (2002). A taxonomy and evaluation of dense two-frame stereo correspondence algorithms. *International Journal of Computer Vision*, 47(1), 7–42.
- [3] OpenCV Development Team (2024). *OpenCV – Open Source Computer Vision Library*. <https://docs.opencv.org/>

- [4] OpenMP Architecture Review Board (2021). *OpenMP Application Program Interface v5.2*. <https://www.openmp.org/>

A Codul sursă complet

A.1 Filtrul Median 5×5

```
1 Mat filtruMedianOptimizat(const Mat& src) {
2     Mat dst = Mat::zeros(src.size(), src.type());
3     int offset = 2; // fereastră 5x5
4
5     #pragma omp parallel for
6     for (int i = offset; i < src.rows - offset; i++) {
7         for (int j = offset; j < src.cols - offset; j++) {
8             uchar valori[25];
9             int index = 0;
10            for (int u = -offset; u <= offset; u++)
11                for (int v = -offset; v <= offset; v++)
12                    valori[index++] = src.at<uchar>(i + u, j + v)
13                    ;
14            sort(valori, valori + 25);
15            dst.at<uchar>(i, j) = valori[12];
16        }
17    }
18    return dst;
19 }
```

Listing 1: Implementarea filtrului Median optimizat (fereastră 5×5).

A.2 Filtrul Gaussian 5×5

```
1 Mat filtruGaussianOptimizat(const Mat& src) {
2     const int kernel[5][5] = {
3         {1, 4, 7, 4, 1},
4         {4, 16, 26, 16, 4},
5         {7, 26, 41, 26, 7},
6         {4, 16, 26, 16, 4},
7         {1, 4, 7, 4, 1}
8     };
9     const int c = 273;
10    Mat dst = Mat::zeros(src.size(), src.type());
11    int offset = 2;
12
13    #pragma omp parallel for
14    for (int i = offset; i < src.rows - offset; i++) {
15        for (int j = offset; j < src.cols - offset; j++) {
16            int suma = 0;
17            for (int u = 0; u < 5; u++)
18                for (int v = 0; v < 5; v++)
19                    suma += kernel[u][v] *
```

```

20         src.at<uchar>(i + u - offset, j + v -
21             offset);
22     dst.at<uchar>(i, j) = static_cast<uchar>(suma / c);
23 }
24 return dst;
25 }

```

Listing 2: Implementarea filtrului Gaussian optimizat (fereastră 5×5).

A.3 Calculul Hărții de Adâncime (SAD)

```

1 Mat calculeazaDepthMapSAD(const Mat& stanga, const Mat& dreapta,
2     int fereastra, int max_disparitate) {
3     Mat depthMap = Mat::zeros(stanga.size(), CV_8UC1);
4     int offset = fereastra / 2;
5
6     #pragma omp parallel for
7     for (int y = offset; y < stanga.rows - offset; y++) {
8         for (int x = offset + max_disparitate;
9             x < stanga.cols - offset; x++) {
10             int min_SAD = INT_MAX, best_disparitate = 0;
11             for (int d = 0; d < max_disparitate; d++) {
12                 int sad_curent = 0;
13                 for (int wy = -offset; wy <= offset; wy++)
14                     for (int wx = -offset; wx <= offset; wx++) {
15                         uchar pS = stanga.at<uchar>(y+wy, x+wx);
16                         uchar pD = dreapta.at<uchar>(y+wy, x+wx-d);
17                         sad_curent += abs(pS - pD);
18                     }
19                 if (sad_curent < min_SAD) {
20                     min_SAD = sad_curent;
21                     best_disparitate = d;
22                 }
23             }
24             depthMap.at<uchar>(y, x) =
25                 static_cast<uchar>((best_disparitate * 255) /
26                     max_disparitate);
27         }
28     }
29     return depthMap;
30 }

```

Listing 3: Algoritmul SAD pentru calculul hărții de adâncime.

A.4 Generarea Anaglifiei 3D

```

1 Mat creeazaAnaglifaCuDepthCorect(const Mat& stanga, const Mat&
    dreapta,

```

```

2         const Mat& depthMap,
3         int max_disparitate) {
4     Mat anaglifa = Mat::zeros(stanga.size(), CV_8UC3);
5     float factor_confort = 0.2f;
6
7     #pragma omp parallel for
8     for (int y = 0; y < stanga.rows; y++) {
9         for (int x = 0; x < stanga.cols; x++) {
10             // Canal Rosu - ochiul stang
11             anaglifa.at<Vec3b>(y, x)[2] = stanga.at<uchar>(y, x);
12
13             int disp = (depthMap.at<uchar>(y, x) *
14                         max_disparitate * factor_confort) / 255;
15             int x_dr = x - disp;
16
17             // Canale Cyan (G+B) - ochiul drept
18             if (x_dr >= 0) {
19                 uchar cyan = dreapta.at<uchar>(y, x_dr);
20                 anaglifa.at<Vec3b>(y, x)[1] = cyan;
21                 anaglifa.at<Vec3b>(y, x)[0] = cyan;
22             } else {
23                 anaglifa.at<Vec3b>(y, x)[1] = 0;
24                 anaglifa.at<Vec3b>(y, x)[0] = 0;
25             }
26         }
27     }
28     return anaglifa;
29 }

```

Listing 4: Sinteza anaglifei 3D sintetice modulate de harta de adâncime.

A.5 Funcția main() – meniu și pipeline principal

Funcția main() gestionează meniul interactiv cu 7 secvențe de test (Spiderman, Rollercoaster, Ball, Helicopter, Bunny, Butterfly, Vettel), citirea și sortarea fișierelor BMP, inițializarea VideoWriter-ului (codec MJPEG, 15 FPS) și bucla cadru-cu-cadru care apelează funcțiile descrise în anexe anterioare. Codul complet este disponibil în fișierul main.cpp din arhiva proiectului.